# Technisches Whitepaper zur horizontalen Skalierung von atlas

*Partition Cells Raft und cloud native Betrieb*

| Repository Basis | Architekturentscheid | Implementierung |
| --- | --- | --- |
| main 67914b24 | ADR 0175 Proposed | Not started |

## Technische Zielarchitektur und Umsetzungspfad

Dieses Whitepaper beschreibt, wie atlas horizontalen Durchsatz und Hochverfügbarkeit erreicht, ohne seine Kerninvarianten aufzugeben. Es übersetzt ADR-0175 in ein implementierbares Gesamtsystem mit klaren Fehlersemantiken, Betriebsrollen, Technologieentscheidungen und Abnahmekriterien.

Version 1.0 | 25. Oktober 2026

# Executive Summary

Die empfohlene Zielarchitektur ist ein Verbund replizierter Partition Cells. Jede logische Partition bleibt eine deterministische Single Writer State Machine, wird im Clusterbetrieb jedoch als eigene Raft Gruppe über mehrere atlas Nodes repliziert. Ein stateless atlas Gateway routet Befehle zum aktuellen Leader. Ein Kubernetes Operator steuert Placement, Rebalancing, Upgrades und Backups. Externe Side Effects bleiben auf getrennten atlas Workern.

Der zentrale Architekturentscheid lautet: Das replizierte Raft Log ist im Clusterbetrieb zugleich der verteilte atlas WAL. Ein Prozessor Batch erzeugt unveränderte atlas Events, repliziert dieses Event Batch als Raft Eintrag und macht es erst nach Quorum Commit sichtbar. Danach wird derselbe Eventstrom wie heute mit genau einem applyToState in Pebble materialisiert. Damit bleiben Event Sourcing, deterministisches Replay, Compile dont interpret und Durable before visible erhalten.

Horizontaler Durchsatz entsteht durch mehr Partition Leader, nicht durch mehr Replikate einer einzelnen Partition. Replikate erhöhen Verfügbarkeit. Eine einzelne Prozessinstanz bleibt bewusst in genau einer Partition; viele unabhängige Instanzen verteilen sich auf viele Partitionen und Nodes. Cross Partition Kommunikation erfolgt asynchron über eine durable Outbox Inbox Envelope. Message Correlation wird durch sharded Directory Partitions gelöst und nicht durch Broadcast oder einen zentralen Flaschenhals.

| Kernaussage | Konsequenz |
|---|---|
| Ein Leader pro Partition | Single Writer und deterministische Reihenfolge bleiben erhalten. |
| Raft Event Batch als Cluster WAL | Eine verteilte Wahrheit statt Raft Log plus zweitem Anwendungs WAL. |
| Stable Partition IDs | Neue Partitionen übernehmen neue Instanzen; bestehende Keys werden nie neu gemappt. |
| Durable Outbox Inbox | At least once Transport mit exactly once Wirkung im Zielzustand. |
| Kubernetes nur als Control Plane | Der API Server liegt nicht im Command oder Event Hot Path. |
| Standalone bleibt erstklassig | Kleine Installationen benötigen weiterhin weder Broker noch SQL Datenbank. |

### Entscheidungsempfehlung

ADR-0175 soll als Leitentscheid beibehalten werden. Die Umsetzung sollte nicht direkt mit verteiltem Consensus beginnen. Zuerst wird eine lokale Multi Partition Fabric gebaut und benchmarked. Danach folgen durable Envelopes und erst dann eine replizierte Partition Cell. Die konkrete Raft Bibliothek bleibt bis zu einem reproduzierbaren Spike offen; etcd raft ist die bevorzugte Referenz, Dragonboat der Vergleichskandidat.

# Inhalt

1 Ausgangslage und Status
2 Ziele und Nichtziele
3 Architekturprinzipien
4 Zielarchitektur atlas Fabric
5 Partition Cells und replizierter Commit
6 Routing Placement und Rebalancing
7 Cross Partition Kommunikation und Correlation
8 Globale Metadaten Artefakte und Abfragen
9 Kubernetes und OpenShift Betriebsmodell
10 Sicherheit und Identität
11 Observability und Betrieb
12 Technologieentscheidung für Raft
13 Umsetzungsprogramm Milestone 5
14 Teststrategie und Korrektheitsnachweis
15 Risiken Trade offs und offene Entscheide
16 Schlussfolgerung
Anhang A Beispiel einer AtlasCluster Ressource
Anhang B Empfohlene Metriken und Alarme
Anhang C Begriffe und Quellen

# 1 Ausgangslage und Status

atlas ist heute eine durable BPMN Engine mit Event Sourcing, lokalem WAL, Pebble Materialisierung und einem Single Writer Processor. Das Datenmodell trägt bereits eine Partition ID; Entity Keys kodieren ihre Partition und der Processor wird mit einer Partition ID erzeugt. Der Server startet auf main dennoch genau eine Partition mit engine.New(1, ...). Das Kubernetes Deployment ist deshalb zu Recht auf eine einzelne stateful Engine Instanz begrenzt.

Der aktuelle Repository Stand vom 25. Oktober 2026 trennt Zielbild und Realität klar. Milestone 5 ist offen. ADR-0175 wurde über Pull Request 512 gemerged, steht aber auf Proposed und Implementation Not started. Dieses Whitepaper ist daher kein Betriebsnachweis, sondern eine technische Entscheidungs und Umsetzungsgrundlage.

| Baustein | Heute implementiert | Zielbild |
|---|---|---|
| Processor | Eine Partition pro Serverprozess | Viele lokale Partitionen pro atlas Node |
| Durability | Lokaler WAL und ein fsync pro Batch | Quorum durable Raft Event Batches |
| State | Lokales Pebble | Eigenes Pebble je Partition Replikat |
| Routing | Lokale API zur Partition 1 | Gateway Routing zum aktuellen Leader |
| Cross Partition | Architekturvorgabe in ADR 0006 | Outbox Inbox Envelopes und Directory Shards |
| High Availability | Nicht implementiert | Raft Replikation und automatischer Failover |
| Kubernetes | Single Replica StatefulSet | Gateway Deployment Node StatefulSet Operator |

Die wichtigste Klarstellung ist wirtschaftlich wie technisch: Replikation und horizontale Skalierung sind zwei verschiedene Eigenschaften. Drei Replikate derselben Partition können einen Node Ausfall tolerieren, erzeugen aber keinen dreifachen Schreibdurchsatz. Mehr Schreibdurchsatz entsteht erst durch mehr unabhängige Partition Leader.

## 1.1 Problemdefinition

- Ein bestätigter Command muss den Ausfall eines Nodes überleben.
- Aggregate Schreiblast muss sich auf mehrere CPU Kerne und Datenträger verteilen.
- Cross Partition Kommunikation darf keine direkte State Abhängigkeit oder globale Sperre einführen.
- Leader Wechsel dürfen keine Side Effects doppelt sichtbar machen und keine committed Followups verlieren.
- Standalone Betrieb muss ohne Kubernetes Broker oder externe Datenbank möglich bleiben.

# 2 Ziele und Nichtziele

| Ziel | Messbarer Nachweis |
|---|---|
| Horizontaler Durchsatz | Aggregate Commands pro Sekunde steigen mit zusätzlichen Partition Leadern bis zu einem dokumentierten Bottleneck. |
| High Availability | Bei RF 3 bleibt ein bestätigter Command nach Ausfall eines Replikats erhalten. |
| Deterministische Recovery | Leader State Follower State und Snapshot plus Log Replay sind identisch. |
| Cloud native Betrieb | Placement Rollout Drain Backup und Repair sind deklarativ über Operator und CRDs steuerbar. |
| Erhalt der Hot Path Invarianten | Keine neue per Command Allocation und keine Runtime Interpretation. |
| Operierbarkeit | Partition Health Lag Commit Apply Snapshot und Outbox Backlog sind sichtbar und alarmierbar. |

## 2.1 Bewusste Nichtziele der ersten Ausbaustufe

- Eine einzelne heisse Prozessinstanz über mehrere Partitionen zu verteilen.
- Laufende Instanzen zwischen logischen Partition IDs zu migrieren.
- Exactly once Ausführung externer Side Effects zu versprechen. Worker bleiben at least once und benötigen Idempotency Keys.
- Synchrone globale Transaktionen oder Two Phase Commit einzuführen.
- Kubernetes zum Teil des Event Commit Protokolls zu machen.
- Den bestehenden Standalone Modus durch einen Cluster Zwang zu ersetzen.

Diese Grenzen sind kein Mangel, sondern schützen die Kernstärken von atlas. Insbesondere bleibt die maximale Geschwindigkeit einer einzelnen Instanz an einen Partition Leader gebunden. Das Produkt skaliert über viele unabhängige Instanzen und nicht durch parallele Mutation derselben Instanz.

# 3 Architekturprinzipien

Die Zielarchitektur wird aus den bestehenden atlas Invarianten abgeleitet. Keine dieser Regeln wird für Clusterbetrieb durch eine alternative Codebahn umgangen.

| Invariante | Konsequenz im Cluster |
|---|---|
| I1 Keine Allokationen im Hot Path | Event Batch Buffer Transport Buffer und Decode Speicher werden wiederverwendet. Multi Raft Scheduling darf nicht pro Command neue Objekte erzeugen. |
| I2 Durable before visible | Der Durability Point ist Quorum Commit. Antworten Notifications Envelopes und Worker Dispatch folgen erst danach. |
| I3 Single Writer | Nur der Leader ordnet Commands. Jedes Replikat wendet committed Events sequenziell auf sein lokales Pebble an. |
| I4 Genau ein applyToState | Leader Follower und Recovery verwenden exakt dieselbe State Mutation. |
| I5 Compile dont interpret | Artefakte sind content addressed verteilt und müssen vor Leadership kompiliert und verifiziert sein. |
| I6 Events sind Fakten | Keys Zeit Routing und ausgewertete Resultate stehen im Event Batch. Followers verarbeiten keine Commands erneut. |

Daraus folgt eine zentrale Designregel: Der verteilte Teil repliziert Ereignisse, nicht Befehle und nicht mutable Prozessobjekte. Ein Command kann abgelehnt werden. Ein committed Event Batch ist dagegen eine geordnete Folge von Fakten und die einzige Grundlage für State und Replay.

# 4 Zielarchitektur atlas Fabric

atlas Fabric ist der Arbeitsname für den Clusterverbund. Er besteht aus vier Runtime Rollen und zwei optionalen Projektionen. Alle Rollen können weiterhin aus demselben Go Binary und demselben Container Image stammen; ihre Trennung ist eine Betriebs und Sicherheitsgrenze, keine Produktaufspaltung.

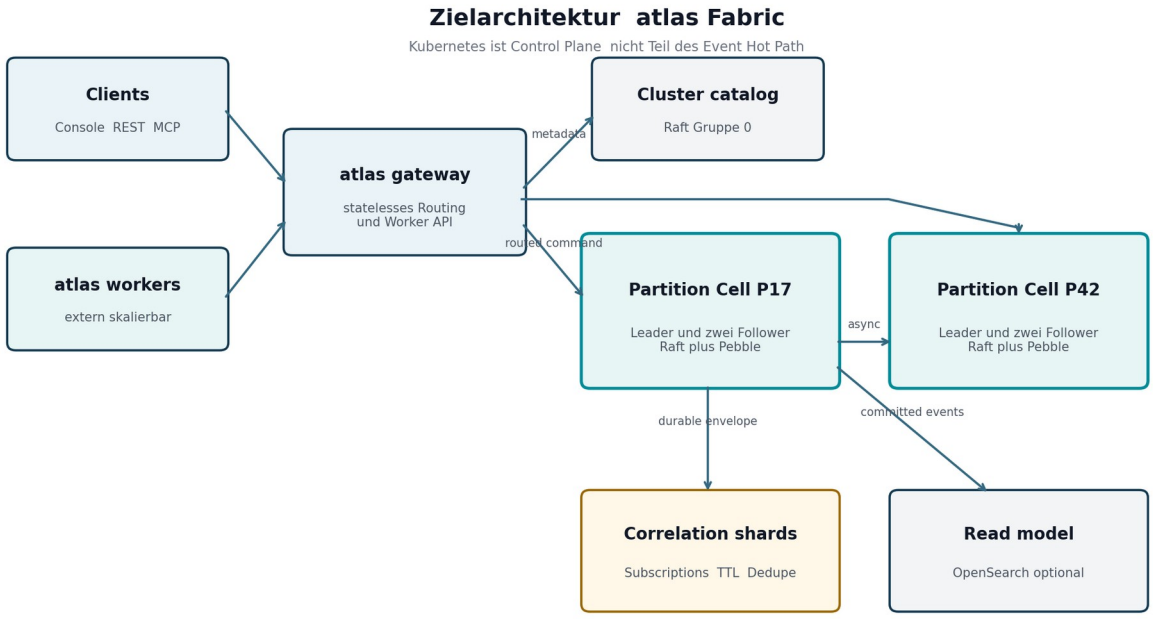



***Abbildung 1 Zielarchitektur mit stateless Gateway replizierten Partition Cells und asynchronem Cross Partition Transport***

| Rolle | Verantwortung | Skalierung |
|---|---|---|
| atlas gateway | REST MCP und Worker API Authentisierung Routing Leader Redirect Handling | Deployment horizontal |
| atlas node | Viele Partition Replikate Raft Transport Pebble Snapshots | StatefulSet operatorgesteuert |
| atlas operator | Placement Membership Rebalancing Upgrade Backup Repair | Deployment mit Leader Election |
| atlas worker | Connectoren Scripts und andere Side Effects ausserhalb der Engine | Deployment nach Backlog |
| Read model | Globale Suche Listen Historie und Analytics | Optional OpenSearch |
| Object store | Immutable Artefakte Checkpoints und Backups | Optional S3 kompatibel |

Der Cluster Catalog ist eine kleine stark konsistente Metadaten Gruppe. Normale Prozess Commands passieren ihn nicht. Damit wird globale Koordination auf seltene Control Plane Operationen begrenzt und der normale Partition Hot Path bleibt lokal.

# 5 Partition Cells und replizierter Commit

## 5.1 Partition Cell

Eine Partition Cell ist die kleinste unabhängig replizierte Ausführungseinheit. Sie besitzt eine stabile Partition ID, eine Raft Gruppe, ein Replica Set, genau einen Leader, einen Processor und pro Replikat eine lokale Pebble Materialisierung. Standard ist ein Replication Factor von drei. Ein Quorum von zwei toleriert damit den Verlust eines Replikats.

| Eigenschaft | Bedeutung |
|---|---|
| Partition ID | Stabil und in Entity Keys kodiert |
| Leader | Einziger Command Ordner und Side Effect Dispatcher |
| Follower | Persistiert Log und materialisiert committed Events |
| Raft Log | Durable Suffix des atlas Event Logs im Clusterbetrieb |
| Pebble | Lokaler abgeleiteter State  niemals zwischen Nodes geteilt |
| Checkpoint | Verifizierter State plus Raft Index Term Epoch Event Position und Artefakt Hashes |

## 5.2 Commit Protokoll

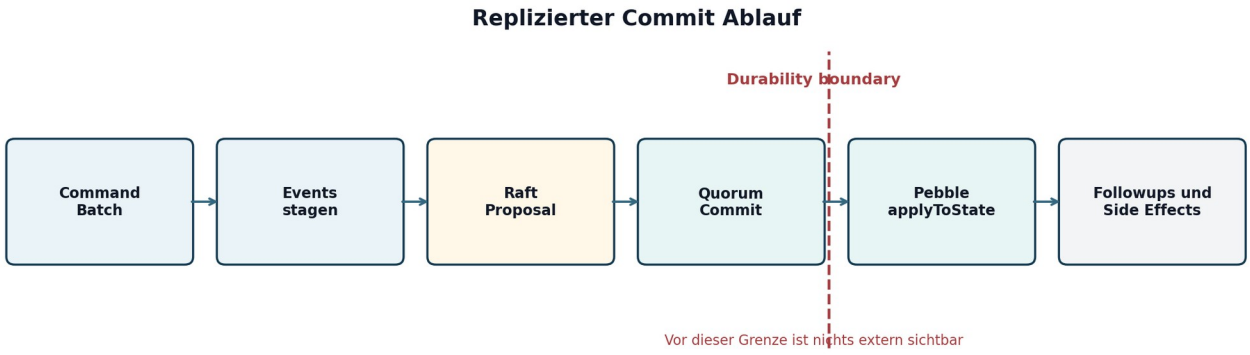



*Abbildung 2  Quorum Commit ersetzt im Clusterbetrieb den lokalen fsync als Durability Point*

1. Das Gateway routet ein Command zum aktuellen Partition Leader.
2. Der Processor erzeugt Events gegen eine noch unsichtbare rollbackfähige Transaktion.
3. Das komplette Event Batch wird als ein Raft Eintrag vorgeschlagen.
4. Die Integration meldet den Eintrag erst als committed wenn er auf einem Quorum dauerhaft gespeichert ist.
5. Leader und Followers wenden die committed Records sequenziell mit demselben applyToState an.
6. Der Leader committed seinen Pebble State und markiert den Applied Index.

7. Erst danach folgen Followup Commands Antworten Notifications Envelopes und Worker Side Effects.

Die erste Implementierung erlaubt pro Partition nur ein in flight Processor Batch. Damit kann kein späteres Batch von spekulativem uncommitted State abhängen. Pipelining ist eine spätere Optimierung und benötigt eine eigene Spec für speculative State und Rollback.

## 5.3 Ein Log statt zwei Wahrheiten

Standalone nutzt weiterhin den heutigen lokalen WAL. Clusterbetrieb nutzt eine atlas eigene CommitLog Abstraktion, deren Raft Einträge die unveränderten atlas Event Records enthalten. Ein separates Anwendungs WAL neben dem Raft Log würde zwei Recovery Cursor zwei fsync Ketten und unklare Crash Semantiken erzeugen. Deshalb ist das Raft Log der Cluster WAL.

```
CommitLog
  AppendEventBatch(batch) -> ProposalID
  AwaitCommit(ProposalID) -> CommitIndex
  Replay(from EventPosition)
  Snapshot(checkpoint)
  Compact(upTo min(checkpoint, consumer watermarks))
```

## 5.4 Recoverable Continuations

Der kritischste noch offene Detailentscheid betrifft Followups. Ein Leader kann nach Quorum Commit aber vor dem Einreihen eines Followup Commands ausfallen. Die neue Führung darf diese Arbeit nicht verlieren. Ein reiner In Memory Queue Eintrag ist deshalb ungenügend.

Empfohlen wird ein expliziter durable Pending Work Index, der aus Fakten im selben Event Batch materialisiert wird. Jeder Eintrag erhält eine deterministische Continuation ID. Die Ausführung ist idempotent und ein erneuter Leader kann offene Einträge wieder einreihen. Wo ein vorhandener transitional State die Arbeit eindeutig ableitbar macht, darf eine spätere ADR eine kompaktere Reconciliation definieren. Persistierte Original Commands und Side Effects in applyToState bleiben ausgeschlossen.

# 6 Routing Placement und Rebalancing

## 6.1 Stable Routing

Partition IDs sind unveränderlich. Ein neuer Prozess wird von der Placement Policy einer aktiven Partition zugewiesen; diese Partition mintet den Instance Key. Der Partition Anteil des Keys bleibt danach die Routing Wahrheit. Eine Modulo Regel instanceKey modulo partitionCount ist verboten, weil eine Änderung der Partition Anzahl bestehende Ownership verschieben würde ohne State zu verschieben.

| Request | Routing Regel |
|---|---|
| Start neue Instanz | Catalog wählt aktive Partition anhand Last und Placement |
| Instance Detail | Partition aus Instance Key |
| Job Completion | Partition aus Job Key |
| Timer Fire | Lokale Partition des Timers |
| Message Publish | Directory Shard aus Tenant Message Name und Correlation Key |
| Globale Suche | Eventual consistent Read Model mit Watermark |

## 6.2 Placement Policy

Der Catalog platziert neue Instanzen gewichtet nach CPU Queue Depth Disk Auslastung und Leader Verteilung. Ein Placement Epoch verhindert, dass ein Gateway mit veralteter Map unbegrenzt schreibt. Bei einem Redirect aktualisiert das Gateway den Cache und wiederholt nur Requests mit stabiler Request ID.

## 6.3 Sichere Replica Bewegung

1. Ein non voting Learner wird auf dem Ziel Node angelegt.
2. Ein verifizierter Snapshot wird übertragen und der Log Suffix nachgezogen.
3. Der Learner erreicht den Applied Index des Leaders und wird über Raft Membership Change zum Voter.
4. Falls notwendig wird Leadership kontrolliert übertragen.
5. Der alte Voter wird aus der committed Konfiguration entfernt.
6. Lokale Daten werden erst danach durch eine getrennte Garbage Collection freigegeben.

Laufende Instanzen werden in der ersten Ausbaustufe nicht zwischen Partition IDs migriert. Rebalancing verschiebt Replikate und Leadership ganzer Partitionen. Das hält Keys und Routing stabil und reduziert den Fehlerraum erheblich.

# 7 Cross Partition Kommunikation und Correlation

## 7.1 Durable Envelopes

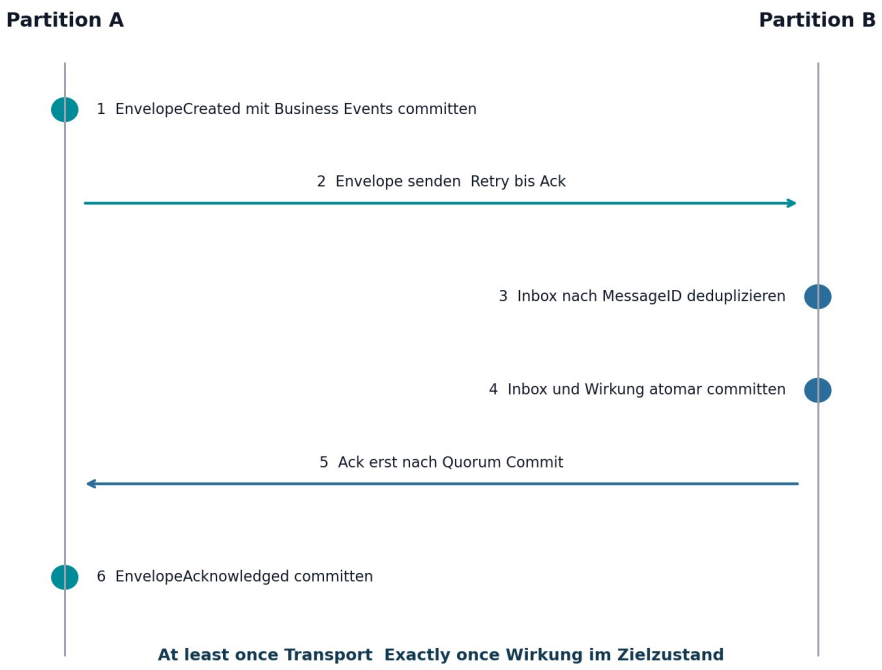



*Abbildung 3 Outbox Inbox Protokoll ohne verteilte Transaktion*

Die Message ID wird stabil aus Source Partition und Source Event Position abgeleitet oder als expliziter Event Wert gespeichert. Transport Wiederholungen sind erlaubt. Die Zielpartition committed Inbox Receipt und fachliche Wirkung atomar. Eine Bestätigung wird erst nach deren Quorum Commit gesendet. Damit entsteht exactly once Wirkung im atlas Zielzustand, nicht exactly once Transport.

| Fehler | Erwartetes Verhalten |
|---|---|
| Sender crasht nach EnvelopeCreated | Neuer Leader liest die Outbox und sendet erneut. |
| Ack geht verloren | Sender sendet erneut Inbox erkennt Message ID. |
| Ziel crasht vor Commit | Kein Ack Retry führt die Wirkung später aus. |
| Ziel crasht nach Commit vor Ack | Retry wird als Duplikat bestätigt ohne zweite Wirkung. |
| Nachrichten werden umgeordnet | Semantik verwendet stabile IDs und wo nötig Source Sequenzen. |

## 7.2 Sharded Correlation Directory

Ein Correlation Key ist nicht zwingend aus dem Instance Key ableitbar und kann erst während der Ausführung entstehen. Deshalb treffen sich Subscription Registrierung und Message Publish deterministisch in demselben Directory Shard.

```
directoryShard = hash(tenantId, messageName, correlationKey) % directoryShardCount
```

Der Shard speichert Subscription Key Zielpartition Instance Key TTL und Dedupe Status. Unmatched Messages können unter einer expliziten TTL gepuffert werden. Nach einem Match erzeugt der Shard eine durable Envelope zur Prozesspartition. Ein Broadcast über alle Execution Partitions ist damit nicht erforderlich.

## 7.3 Call Activities

Parent und Child werden zunächst in derselben Partition platziert. Cross Partition Call Activities sind eine spätere Ausbaustufe, weil Parent Child Completion Cancellation und Compensation über Node Ausfälle hinweg ein eigenes Protokoll benötigen. Die Envelope Infrastruktur ist Voraussetzung, aber noch keine vollständige Semantik.

# 8 Globale Metadaten Artefakte und Abfragen

## 8.1 Cluster Catalog

Raft Gruppe 0 verwaltet kleine stark konsistente Metadaten. Dazu gehören Partition Map und Placement Epochs, Process Definition Keys und Versionen, Artifact Hashes, Cluster Feature Version, Wire Format Version und die Registry der Worker Types. Sie koordiniert Deployment und Betrieb, nicht normale Token Bewegung.

## 8.2 Immutable Artefakte

BPMN DMN Forms und grosse Checkpoints können in einem S3 kompatiblen Object Store liegen. Inhalte werden per kryptografischem Hash adressiert. Bevor ein Node Leader werden darf, muss er alle benötigten Artefakte geladen, den Hash verifiziert und mit einer kompatiblen Compiler Version vorkompiliert haben. Runtime Download oder XML Parsing beim Token Eintritt ist ausgeschlossen.

## 8.3 Reads und globale Projektionen

| Read Typ | Quelle | Konsistenz |
|---|---|---|
| Instance oder Job Detail | Owning Partition Leader | Linearizable oder leader consistent |
| Follower Read | Follower nach ReadIndex und Applied Wait | Explizit stark oder explizit stale |
| Globale Listen | OpenSearch oder austauschbare Projektion | Eventual consistent mit Watermark |
| Audit und Export | Committed Event Stream pro Partition | Idempotent über Partition plus Position |
| Metriken | Prometheus Endpoint | Operational nicht transaktional |

Jede globale Antwort soll ihren Index Watermark ausweisen. Die Console kann dadurch sichtbar machen, ob eine Liste den aktuellen Stand oder einen wenige Sekunden älteren Projektionsstand zeigt. Detailansichten springen immer zur autoritativen Partition zurück.

# 9 Kubernetes und OpenShift Betriebsmodell

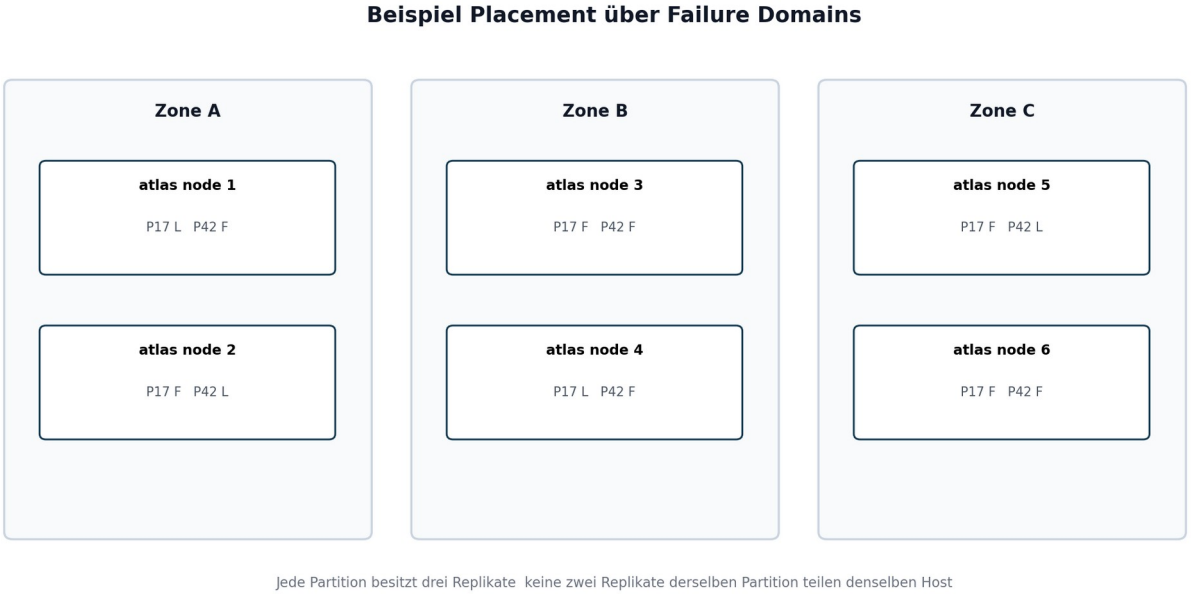



*Abbildung 4 Beispielhafte Verteilung von Partition Replikaten über Nodes und Zonen*

| Kubernetes Objekt | Verwendung |
|---|---|
| Deployment atlas gateway | Stateless API Replikate hinter Service oder Ingress |
| StatefulSet atlas node | Stabile Node Identity und eigene RWO PVC je Pod |
| Headless Service | Peer Discovery und stabile DNS Namen |
| Deployment atlas operator | CRD Reconciliation mit eigener Leader Election |
| Deployments atlas worker | Getrennte Worker Pools nach Trust Zone oder Connector Art |
| PodDisruptionBudget | Schützt Quorum vor freiwilligen gleichzeitigen Evictions |
| Topology Spread und Anti Affinity | Verteilt Replikate über Hosts und Zonen |
| NetworkPolicy | Erlaubt nur definierte Gateway Node Worker und Operator Pfade |

Kubernetes kennt Pods und Volumes, aber nicht den committed Raft Membership Zustand. Deshalb darf ein HPA Gateways und Worker skalieren, jedoch keine atlas Nodes blind verkleinern. Node Scale Down wird ausschliesslich vom Operator ausgeführt, nachdem alle Partition Replikate quorum sicher evakuiert wurden.

## 9.1 Operator Reconciliation

- Desired Node Count und Storage Class bereitstellen.
- Partition Replikate gemäss Replication Factor und Failure Domains platzieren.
- Under replicated oder lagging Cells erkennen und reparieren.
- Membership Changes einzeln und idempotent durchführen.
- Rolling Upgrades mit Version Gates und Leadership Transfer steuern.
- Backup Epochs und Restore Workflows koordinieren.
- Destruktive Aktionen durch Conditions Finalizers und explizite Freigaben schützen.

## 9.2 Backup und Restore

Ein Cluster Backup erhält eine Backup Epoch. Jede Partition erstellt an einem committed Cut einen verifizierten Checkpoint inklusive Outbox Inbox und Applied Index. Erst wenn alle benötigten Partitionen und der Catalog erfolgreich publiziert sind, wird ein unveränderliches Cluster Manifest geschrieben. Restore validiert Hashes Format und Compiler Kompatibilität bevor irgendeine Partition traffic ready wird.

# 10 Sicherheit und Identität

Scale out vergrössert die Angriffsoberfläche. Die Sicherheitsarchitektur muss deshalb als Bestandteil des Protokolls und nicht als später Reverse Proxy verstanden werden.

| Grenze | Kontrolle |
|---|---|
| Gateway zu Node | mTLS Service Identity Authorization je RPC Request ID und Rate Limit |
| Node zu Node | mTLS Peer Identity zugelassene Cluster ID Term und Replay Schutz |
| Worker zu Gateway | OIDC oder kurzlebiges Worker Credential Scopes je Job Type |
| Operator zu Kubernetes | Eigener ServiceAccount minimaler RBAC Scope |
| Artefakt Store | TLS server side encryption immutable keys Hash Verification |
| Backups | Verschlüsselung getrennte Credentials Retention und Restore Audit |

Connector Credentials verbleiben gemäss ADR-0168 beim Worker. Sie erscheinen weder im BPMN Modell noch in Events noch im Raft Log. Der Engine Node löst nur kompilierten Task Detail und Variablen in ein portables Job Payload auf.

## 10.1 Fencing

Nur der aktuelle Leader darf Notifications Envelopes oder Job Leases dispatchen. Jeder Dispatcher wird mit Raft Term oder monotoner Leadership Epoch gefenced. Eine Job Completion trägt Lease Token und Partition Epoch. Ein ehemaliger Leader oder ein verspäteter Worker darf damit keinen neueren Zustand überschreiben.

## 10.2 Supply Chain und Versionen

Cluster Releases müssen Wire Format Event Format Snapshot Format und Compiler Version explizit deklarieren. Rolling Upgrades erlauben nur dokumentierte Nachbarschaften, beispielsweise N und N minus 1. Feature Bits werden erst aktiviert, wenn alle Voter sie verstehen. Container Images werden signiert und per Digest referenziert; CRDs und Webhooks müssen upgrade und downgrade Regeln besitzen.

# 11 Observability und Betrieb

Ein verteilter Workflow Cluster ist nur betreibbar, wenn Consensus State und fachlicher State gemeinsam sichtbar sind. Prometheus erfasst dimensionale Metriken, OpenTelemetry verbindet Gateway Routing Raft Proposal Apply und Worker Completion in einem Trace. Logs enthalten stabile Request und Message IDs, aber keine Variablen oder Credentials per Default.

| Bereich | Schlüsselindikatoren |
|---|---|
| Consensus | Leader Term Elections Proposal Latency Commit Index Quorum Health |
| Apply | Applied Index Commit Apply Lag Batch Size applyToState Duration |
| Partition | Command Queue Depth Rejections Hot Partition Load Disk Bytes |
| Transport | Outbox Backlog Retry Age Inbox Duplicates Correlation Waits |
| Snapshots | Checkpoint Age Transfer Bytes Verification Failures Restore Duration |
| Workers | Lease Age Completion Latency Retries Idempotency Conflicts |
| Projection | Exporter Watermark Lag Index Failures Query Freshness |

## 11.1 Empfohlene initiale SLOs

| SLO | Vorschlag | Hinweis |
|---|---|---|
| Durability | RPO 0 für bestätigte Commands bei einem Node Ausfall mit RF 3 | Protokollgarantie |
| Availability | 99.9 Prozent Write API pro Monat als erste Produktionsstufe | Nach Failover Tests schärfen |
| Failover | p95 kleiner 10 Sekunden im gleichen Cluster | Messziel kein ADR Fakt |
| Projection freshness | p95 kleiner 5 Sekunden | Nur globale Read Models |
| Outbox age | p99 kleiner 30 Sekunden ohne Zielstörung | Alert bei ansteigendem Trend |

Absolute Durchsatz und Latenzziele werden erst nach M5.0 und M5.2 festgelegt. Vorher wären Zahlen Marketing statt Engineering. Der wichtigste Performance Nachweis ist die Skalierungseffizienz: mehr Partition Leader müssen den Gesamtdurchsatz bis zu einem klar gemessenen gemeinsamen Bottleneck erhöhen.

# 12 Technologieentscheidung für Raft

ADR-0175 entscheidet die Multi Raft Architektur, nicht die Bibliothek. Diese Trennung ist richtig: Die Auswahl beeinflusst Storage Transport Snapshot Semantik Scheduling und Upgrade Fähigkeit und muss durch einen atlas spezifischen Spike belegt werden.

| Kriterium | etcd raft | Dragonboat |
|---|---|---|
| Modell | Raft Core als deterministische State Machine | Integrierte Multi Group Runtime |
| atlas Kontrolle | Sehr hoch  Transport Storage Snapshots selbst | Mittel  mehr Runtime Entscheidungen in Library |
| Multi Raft | Muss atlas selbst schedulen | Nativ |
| Integration | Mehr Engineering aber klare CommitLog Grenze | Schneller Spike möglich |
| Snapshot und Membership | Primitive vorhanden  Orchestrierung bei atlas | Umfangreich integriert |
| Abhängigkeitsrisiko | Kleine stabile Core API | Grössere Kopplung an Runtime und Storage |
| Empfehlung | Bevorzugte Referenz für Produktion | Benchmark und Produktivitätsvergleich |

etcd raft ist die bevorzugte Richtung, weil atlas sein Event Format sein Snapshot Format und seine Hot Path Kontrolle behalten kann. Dragonboat soll dennoch im Spike mit identischen Workloads gemessen werden. Es bietet Multi Group Funktionalität und kann Entwicklungsrisiko reduzieren; der Gewinn muss aber gegen Dependency und Upgrade Kopplung abgewogen werden.

## 12.1 Spike Abnahmekriterien

- Drei Prozesse replizieren dieselben atlas Event Batches und bestehen Leader Kill Tests.
- Live Leader Follower und Snapshot plus Log Replay State sind byte oder semantisch identisch.
- Allocation und CPU Profil des Processor Hot Path bleiben innerhalb eines definierten Budgets.
- Membership Change Snapshot Transfer ReadIndex und Leadership Transfer werden demonstriert.
- Mixed Version und Wire Format Verhalten ist dokumentiert.
- Recovery bei Disk full corrupt snapshot slow follower und Netzwerkpartition ist fail closed.
- Throughput Latenz und fsync Verhalten werden mit atlas typischen Batch Grössen verglichen.

## 12.2 Nicht empfohlene primäre Lösungen

| Alternative | Warum nicht als Engine Wahrheit |
|---|---|
| Kafka oder Redpanda | Externe Pflichtabhängigkeit und schwierige Atomizität zwischen Log und Pebble. Sehr gut als Export Sink. |
| NATS JetStream | Nützlich für optionale Integration aber nicht nötig im internen Commit Hot Path. |
| PostgreSQL oder CockroachDB | Ersetzt die log strukturierte Engine durch Datenbank Transaktionen und verändert das Performance Modell. |
| Shared PVC oder NFS | Kein authoritative Writer Protokoll und Risiko gleichzeitiger Pebble Öffnung. |
| Kubernetes Lease | Wählt einen Leader repliziert aber weder Event Historie noch committed State. |

# 13 Umsetzungsprogramm Milestone 5

Die Umsetzung erfolgt als vertikale Slices. Jede Stufe liefert einen lauffähigen und testbaren Zustand; Standalone bleibt nach jeder Stufe grün. Netzwerk und Consensus kommen erst hinzu, nachdem lokale Partitionierung und Recovery nachweislich korrekt sind.

| Phase | Lieferumfang | Exit Gate |
|---|---|---|
| M5.0 Local Partition Fabric | PartitionManager Router getrennte Directories mehrere Processor Goroutines | Nahezu lineare Multi Core Skalierung und Recovery je Partition |
| M5.1 Durable Envelopes | Outbox Inbox Message IDs Retry Ack Directory Shards | Duplicate Delay Reorder und Crash Tests mit zwei lokalen Partitionen |
| M5.2 Replicated Cell | CommitLog Adapter drei Prozesse Snapshot Failover | Quorum Durability und Leader Follower Replay Equivalence |
| M5.3 Cluster Gateway | Leader Routing Worker Multiplexing mTLS Backpressure | End to end Jobs Reads Writes und Failover |
| M5.4 Kubernetes Operator | CRD Placement Membership Upgrade Drain Repair | Quorum safe Rolling Upgrade und Node Evacuation |
| M5.5 Backup Rebalancing Read Model | Learner Transfer Cluster Manifest OpenSearch Watermarks | Restore Drill Rebalance und globale Operations Views |

## 13.1 Empfohlene ADR Folge

1. CommitLog boundary and quorum durability semantics
2. Partition runtime placement epochs and request routing
3. Recoverable continuations and durable pending work
4. Cross partition envelope wire format and deduplication
5. Sharded correlation directory and BPMN message races
6. Consensus library selection based on the spike
7. Snapshot compaction exporter watermarks and cluster backup
8. Leadership fencing worker leases and idempotency contracts
9. Cluster compatibility rolling upgrades and feature gates

## 13.2 Team und Governance

Der Scale out Strang sollte als eigenes Architekturprogramm mit einem klaren Owner geführt werden. Code Reviews benötigen mindestens eine zweite Person mit Distributed Systems Erfahrung. Jede Phase besitzt eine Failure Matrix Benchmarks eine Recovery Suite und eine aktualisierte Betriebsdokumentation. Feature Fortschritt wird nicht an CRDs oder Netzwerkcode gemessen, sondern an bestandenen Korrektheits Gates.

# 14 Teststrategie und Korrektheitsnachweis

Distributed Correctness entsteht nicht aus Happy Path Integration Tests. atlas benötigt drei komplementäre Ebenen: deterministische State Machine Tests, simulierte Netzwerk und Clock Tests sowie Black Box Fault Injection auf echten Prozessen.

| Ebene | Werkzeug | Nachweis |
|---|---|---|
| Unit und Property | Go Tests Fake Clock generierte Eventfolgen | Idempotency Ordering Codec und applyToState |
| Deterministische Simulation | In Memory Transport steuerbare Delivery und Elections | Delay Drop Duplicate Reorder Partition ohne Flakiness |
| Recovery | Persist Restart Replay Compare | Live State entspricht Replay State für jede neue Event Art |
| Multi Process | Test Harness mit Prozess Kill Disk und Network Faults | Leader Failover Quorum Membership Snapshot |
| System Linearizability | History Checker für Write und Read Operationen | Keine bestätigten verlorenen oder widersprüchlichen Writes |
| Performance | Benchmarks mit benchmem p50 p95 p99 und Profilen | Bounded Allocations und Skalierungseffizienz |

## 14.1 Obligatorische Failpoints

- Leader stoppt vor Proposal nach lokaler Persistenz nach Quorum Commit nach Apply und vor Side Effect.
- Leader wird vom Quorum getrennt und versucht weiter zu dispatchen.
- Follower ist langsam voll oder liefert einen korrupten Snapshot.
- Ack und Envelope werden beliebig dupliziert verzögert und umgeordnet.
- Operator startet während Membership Change oder Snapshot Transfer neu.
- Node Drain und Rolling Upgrade treffen auf eine bereits degradierte Partition.
- Object Store ist während Backup und Restore teilweise nicht erreichbar.
- Projection Exporter fällt zurück und darf WAL Compaction nicht über seinen Watermark hinaus erlauben.

## 14.2 Definition of Done für Cluster Slices

- go build ./... und go test ./... sind grün.
- go test -race -timeout=45m ./... ist grün.
- go vet ./... ist grün und gofmt -l . bleibt leer.
- Persistence Processor Event Job Timer Message oder Variable Änderungen besitzen Recovery Tests.
- Neue Hot Path Arbeit besitzt Allocation Benchmarks.
- Failure Semantik und Operator Runbook sind dokumentiert.
- Die Abweichung zwischen implementiertem Stand ADR und Roadmap ist aktualisiert.

# 15 Risiken Trade offs und offene Entscheide

| Risiko | Auswirkung | Mitigation | Priorität |
|---|---|---|---|
| Verlorene Continuation nach Commit | Prozess bleibt hängen | Durable Pending Work und Crash Tests | Kritisch |
| Stale Leader dispatcht Side Effects | Doppelte externe Wirkung | Term Epoch Fencing und Idempotency Key | Kritisch |
| Raft und atlas Log Semantik divergieren | Recovery Inkonsistenz | Ein CommitLog und ein Snapshot Manifest | Kritisch |
| Hot Partition | Einzelne Queue dominiert Latenz | Load aware Placement Limits und spätere Split Strategie | Hoch |
| Catalog wird Bottleneck | Deploy und Placement stocken | Nicht im Execution Hot Path kleine Entries Caches | Mittel |
| Operator entfernt Quorum | Partition nicht verfügbar | Learner Workflow Finalizers und Safety Checks | Kritisch |
| Mixed Version Fehler | Cluster Split oder inkompatibler State | Feature Gates und explizite Format Versionen | Hoch |
| Projection Lag wird als aktuell gelesen | Falsche Operations Sicht | Watermarks und direkte Detail Reads | Mittel |

## 15.1 Noch zu treffende Produktentscheide

- Ist Clusterbetrieb ein separates Packaging oder derselbe Binary Modus mit Lizenz und Feature Flag?
- Welche Konsistenz wird für welche Read API garantiert und wie wird stale explizit bezeichnet?
- Welche S3 Implementierungen und Storage Classes werden offiziell supportet?
- Welche Maximalwerte gelten für Partitionen Replikate Event Batch und Snapshot Grösse?
- Wie lange werden unmatched Messages und Dedupe IDs aufbewahrt?
- Welche Upgrade Nachbarschaften und Downgrade Grenzen sind vertraglich garantiert?

## 15.2 Akzeptierte Trade offs

Consensus erhöht Write Latenz und verdreifacht bei RF 3 grob die Log Storage und Netzwerkarbeit. Cross Partition Prozesse werden eventual consistent. Der Operator erweitert den Wartungsumfang deutlich. Diese Kosten sind akzeptabel, weil sie explizit Verfügbarkeit und horizontalen Durchsatz kaufen, während die standalone Ausführung ihre heutige Einfachheit behält.

# 16 Schlussfolgerung

Die richtige horizontale Skalierung für atlas ist keine Datenbank Migration und kein Broker vor der Engine. Sie ist die verteilte Fortsetzung der bestehenden Architektur: viele unabhängige Single Writer Partitionen, deren Eventstrom repliziert und deren State deterministisch materialisiert wird.

Partition Cells trennen sauber zwischen Durchsatz und Verfügbarkeit. Mehr Cells liefern mehr Leader und damit mehr parallele Schreibkapazität. Mehr Replikate liefern Fehlertoleranz. Durable Envelopes und Directory Shards lösen Cross Partition Semantik ohne globale Transaktion. Ein Operator macht Placement und Lifecycle cloud native, ohne Kubernetes in die fachliche Commit Kette zu ziehen.

Der unmittelbar nächste Schritt ist M5.0: ein lokaler PartitionManager mit vier unabhängigen Partitionen, getrennten Stores und einem Router. Erst wenn Skalierung, Recovery und Hot Path Budget dort belegt sind, folgt M5.1 mit Envelopes und danach M5.2 mit einem reproduzierbaren etcd raft gegen Dragonboat Spike. Dieses Vorgehen minimiert gleichzeitig Architektur und Implementierungsrisiko.

## Empfohlener Beschluss

ADR-0175 bleibt der Leitentscheid für Milestone 5. Die Umsetzung startet mit der Local Partition Fabric. Jede folgende Phase wird erst nach bestandenen Recovery Fault Injection und Performance Gates freigegeben.

# A Beispiel einer AtlasCluster Ressource

Die folgende CRD Form ist bewusst illustrativ. Felder und API Version werden erst mit dem Operator ADR stabilisiert.

```
apiVersion: atlas.blumer.cloud/v1alpha1
kind: AtlasCluster
metadata:
  name: production
spec:
  nodes: 5
  partitions: 64
  replicationFactor: 3
  storage:
    size: 200Gi
    storageClass: fast-rwo
  placement:
    spreadBy:
      - kubernetes.io/hostname
      - topology.kubernetes.io/zone
  backup:
    url: s3://atlas-backups/production
    schedule: "0 */6 * * *"
  security:
    mtls: required
  observability:
    prometheus: true
    openTelemetry: true
```

| Condition | Bedeutung |
|---|---|
| Available | Alle kritischen Services bereit und Catalog Quorum vorhanden |
| PartitionsHealthy | Alle Partitionen besitzen Quorum und akzeptablen Apply Lag |
| Rebalancing | Mindestens eine Replica Bewegung läuft |
| UpgradeBlocked | Version oder Snapshot Kompatibilität verhindert Rollout |
| BackupDegraded | Letzter Cluster Manifest Abschluss fehlgeschlagen |

# B Empfohlene Metriken und Alarme

| Metrik | Labels | Alarmidee |
|---|---|---|
| atlas_raft_proposal_seconds | node partition result | p99 über Ziel oder Error Rate |
| atlas_raft_commit_apply_lag | node partition | Lag steigt für mehrere Intervalle |
| atlas_partition_command_queue | partition | Queue über Kapazitätsgrenze |
| atlas_partition_leader | node partition | Kein Leader oder Election Flapping |
| atlas_replica_match_index_lag | node partition peer | Follower kann nicht aufholen |
| atlas_outbox_oldest_seconds | source partition target | Älteste Envelope über SLO |
| atlas_correlation_unmatched | directory shard message | Unmatched Wachstum oder TTL Expiry |
| atlas_checkpoint_age_seconds | partition | Kein verifizierter Checkpoint |
| atlas_snapshot_transfer_bytes | source target partition | Stall oder wiederholter Restart |
| atlas_projection_watermark_lag | projection partition | Globale Sicht zu alt |

Labels mit ungebundener Kardinalität wie Instance Key Job Key Correlation Key oder Tenant Name gehören nicht in Prometheus Metriken. Solche Details werden über Traces Logs und die autoritative Operations API untersucht.

# C Begriffe und Quellen

## C.1 Begriffe

| Begriff | Definition |
|---|---|
| Partition Cell | Logische atlas Partition mit Raft Gruppe Leader Replikaten Event Log und lokaler State Materialisierung |
| Commit Index | Höchster Raft Log Index der durch ein Quorum committed ist |
| Applied Index | Höchster committed Index den das lokale applyToState verarbeitet hat |
| Envelope | Durable Cross Partition Nachricht mit stabiler Message ID |
| Directory Shard | Partitionierter Index für Message Subscriptions und wartende Publications |
| Placement Epoch | Monotone Version der Partition Map zur Erkennung veralteten Routings |
| Learner | Non voting Replica während Snapshot und Catch up |
| Watermark | Bis zu welcher Event Position ein Consumer sicher verarbeitet hat |

## C.2 Repository Quellen

| ID | Quelle |
|---|---|
| **R1** | atlas Repository main commit 67914b24 |
| **R2** | ADR 0175 Replicated partition cells for horizontal scale out |
| **R3** | atlas Invariants |
| **R4** | atlas Architecture |
| **R5** | atlas Roadmap Milestone 5 |
| **R6** | ADR 0002 Single writer partition model |
| **R7** | ADR 0005 Group commit and fsync strategy |
| **R8** | ADR 0006 Partition routing and cross partition communication |
| **R9** | ADR 0131 Recovery checkpoints and WAL compaction |

| ID | Quelle |
|---|---|
| R10 | ADR 0168 Connector work on a worker |
| R11 | Pull Request 512 |

## C.3 Externe Primärquellen

| ID | Quelle |
|---|---|
| E1 | Raft paper  Ongaro und Ousterhout |
| E2 | etcd raft |
| E3 | Dragonboat multi group Raft |
| E4 | Kubernetes StatefulSets |
| E5 | Kubernetes Operator pattern |
| E6 | Kubernetes topology spread constraints |
| E7 | Kubernetes PodDisruptionBudget |
| E8 | Prometheus data model |
| E9 | OpenTelemetry signals |
| E10 | Pebble |